

Publish an Android app using the command line

Article • 09/24/2024

To distribute a .NET Multi-platform App UI (.NET MAUI) Android app, you'll need to sign it with a key from your keystore. A *keystore* is a database of security certificates that's created by using `keytool` from the Java Development Kit (JDK). A keystore is required when publishing a .NET MAUI Android app, as Android won't run apps that haven't been signed.

Create a keystore file

During development, .NET for Android uses a debug keystore to sign the app, which allows it to be deployed directly to an emulator or to devices configured to run debuggable apps. However, this keystore isn't recognized as a valid keystore for the purposes of distributing apps. Therefore, a private keystore must be created and used for signing release builds. This is a step that should only be performed once, as the same key will be used for publishing updates and can be used to sign other apps. After generating a keystore file, you'll supply its details from the command line when building the app, or configure your project file to reference it.

Perform the following steps to create a keystore file:

1. Open a terminal and navigate to the folder of your project.

Tip

If Visual Studio is open, use the **View > Terminal** menu to open a terminal at the location of the solution or project. Navigate to the project folder.

2. Run the `keytool` tool with the following parameters:

Console

```
keytool -genkeypair -v -keystore {filename}.keystore -alias {key-name} -keyalg RSA -keysize 2048 -validity 10000
```

Important

If you have multiple versions of the JDK installed on your computer, ensure that you run `keytool` from the latest version of the JDK.

You'll be prompted to provide and confirm a password, followed by your full name, organization unit, organization, city or locality, state or province, and country code. This information isn't displayed in your app, but is included in your certificate.

For example, to generate a *myapp.keystore* file in the same folder as your project, with an alias of `myapp`, use the following command:

Console

```
keytool -genkeypair -v -keystore myapp.keystore -alias myapp -  
keyalg RSA -keysize 2048 -validity 10000
```

Tip

Backup your keystore and password. If you lose it you'll be unable to sign your app with the same signing identity.

Find your keystore's signature

To list the keys that are stored in a keystore, use `keytool` with the `-list` option:

Console

```
keytool -list -keystore {filename}.keystore
```

For example, to list the keys in a keystore named *myapp.keystore*, use the following command:

Console

```
keytool -list -keystore myapp.keystore
```

Build and sign your app

To build your app from the command line, and sign it using your keystore, open a terminal and navigate to the folder for your .NET MAUI app project. Run the `dotnet publish` command, providing the following parameters:

[Expand table](#)

Parameter	Value
<code>-f</code> or <code>--framework</code>	The target framework, which is <code>net8.0-android</code> .
<code>-c</code> or <code>--configuration</code>	The build configuration, which is <code>Release</code> .

 Warning

Attempting to publish a .NET MAUI solution will result in the `dotnet publish` command attempting to publish each project in the solution individually, which can cause issues when you've added other project types to your solution. Therefore, the `dotnet publish` command should be scoped to your .NET MAUI app project.

Additional build parameters can be specified on the command line, if they aren't provided in a `<PropertyGroup>` in your project file. The following table lists some of the common parameters:

[Expand table](#)

Parameter	Value
<code>-p:ApplicationTitle</code>	The user-visible name for the app.
<code>-p:ApplicationId</code>	The unique identifier for the app, such as <code>com.companyname.mymauiapp</code> .
<code>-p:ApplicationVersion</code>	The version of the build that identifies an iteration of the app.
<code>-p:ApplicationDisplayVersion</code>	The version number of the app.
<code>-p:AndroidKeyStore</code>	A boolean value that indicates whether the app should be signed. The default value is <code>false</code> .
<code>-p:AndroidPackageFormats</code>	A semi-colon delimited property that indicates if you want to package the app as an APK file or AAB. Set to either <code>aab</code> or <code>apk</code> to generate only one format. The default value for release builds is <code>aab;apk</code> .
<code>-p:AndroidSigningKeyAlias</code>	The alias for the key in the keystore. This is the <code>keytool -alias</code> value used when creating the keystore.
<code>-p:AndroidSigningKeyPass</code>	The password of the key within the keystore file. This is the value provided to <code>keytool</code> when creating the keystore file and you're asked to enter the keystore password. This is because

Parameter	Value
	the default keystore type assumes that the key password and keystore password are identical. This property also supports <code>env:</code> and <code>file:</code> prefixes that can be used to specify an environment variable or file that contains the password. These options provide a way to prevent the password from appearing in build logs.
<code>-p:AndroidSigningKeyStore</code>	The filename of the keystore file created by <code>keytool</code> . This is the <code>keytool -keystore</code> value used when creating the keystore.
<code>-p:AndroidSigningStorePass</code>	The password for the keystore file. This is the value provided to <code>keytool</code> when creating the keystore file and you're asked to enter the keystore password. This is because the default keystore type assumes that the keystore password and key password are identical. This property also supports <code>env:</code> and <code>file:</code> prefixes that can be used to specify an environment variable or file that contains the password. These options provide a way to prevent the password from appearing in build logs.
<code>-p:PublishTrimmed</code>	A boolean value that indicates whether the app should be trimmed. The default value is <code>true</code> for release builds.

You should use the same password as the values of the `AndroidSigningKeyPass` and `AndroidSigningStorePass` parameters.

For a full list of build properties, see [Build properties](#).

Important

Values for these parameters don't have to be provided on the command line. They can also be provided in the project file. When a parameter is provided on the command line and in the project file, the command line parameter takes precedence. For more information about providing build properties in your project file, see [Define build properties in your project file](#).

Run the `dotnet publish` command with the following parameters to build and sign your app:

Console

```
dotnet publish -f net8.0-android -c Release -p:AndroidKeyStore=true -  
p:AndroidSigningKeyStore={filename}.keystore -p:AndroidSigningKey-
```

```
Alias={keyname} -p:AndroidSigningKeyPass={password} -p:AndroidSigningStorePass={password}
```

❗ Note

In .NET 8, the `dotnet publish` command defaults to the `Release` configuration. Therefore, the build configuration can be omitted from the command line.

For example, use the following command to build and sign your app using the previously created keystore:

Console

```
dotnet publish -f net8.0-android -c Release -p:AndroidKeyStore=true -p:AndroidSigningKeyStore=myapp.keystore -p:AndroidSigningKeyAlias=myapp -p:AndroidSigningKeyPass=mypassword -p:AndroidSigningStorePass=mypassword
```

Both the `AndroidSigningKeyPass` and `AndroidSigningStorePass` properties support `env:` and `file:` prefixes that can be used to specify an environment variable or file that contains the password. Specifying the password in this way prevents it from appearing in build logs. For example, to use an environment variable named `AndroidSigningPassword`:

Console

```
dotnet publish -f net8.0-android -c Release -p:AndroidKeyStore=true -p:AndroidSigningKeyStore=myapp.keystore -p:AndroidSigningKeyAlias=myapp -p:AndroidSigningKeyPass=env:AndroidSigningPassword -p:AndroidSigningStorePass=env:AndroidSigningPassword
```

❗ Important

The `env:` prefix isn't supported when `$(AndroidPackageFormat)` is set to `aab`.

To use a file located at `C:\Users\user1\AndroidSigningPassword.txt`:

Console

```
dotnet publish -f net8.0-android -c Release -p:AndroidKeyStore=true -p:AndroidSigningKeyStore=myapp.keystore -p:AndroidSigningKeyAlias=myapp -p:AndroidSigningKeyPass=file:C:\Users\user1\AndroidSigningPass-
```

```
word.txt -
p:AndroidSigningStorePass=file:C:\Users\user1\AndroidSigningPass-
word.txt
```

Publishing builds and signs the app, and then copies the AAB and APK files to the *bin\Release\net8.0-android\publish* folder. There are two AAB files - one unsigned and another signed. The signed variant has **-signed** in the file name.

For more information about the `dotnet publish` command, see [dotnet publish](#).

ⓘ **Note**

For Android apps, `dotnet build` can also be used to build and sign your app. However, AAB and APK files will be created in the *bin\Release\net8.0-android* folder rather than the *publish* subfolder. `dotnet build` also defaults to a `Debug` configuration, so the `-c` parameter is required to specify the `Release` configuration.

Define build properties in your project file

An alternative to specifying build parameters on the command line is to specify them in your project file in a `<PropertyGroup>`. The following table lists some of the common build properties:

 **Expand table**

Property	Value
<code><ApplicationTitle></code>	The user-visible name for the app.
<code><ApplicationId></code>	The unique identifier for the app, such as <code>com.companyname.mymauiapp</code> .
<code><ApplicationVersion></code>	The version of the build that identifies an iteration of the app.
<code><ApplicationDisplayVersion></code>	The version number of the app.
<code><AndroidKeyStore></code>	A boolean value that indicates whether the app should be signed. The default value is <code>false</code> .
<code><AndroidPackageFormats></code>	A semi-colon delimited property that indicates if you want to package the app as an APK file or AAB. Set to either <code>aab</code> or <code>apk</code> to generate only one format. The default value for release builds is <code>aab;apk</code> .

Property	Value
<code><AndroidSigningKeyAlias></code>	The alias for the key in the keystore. This is the <code>keytool -alias</code> value used when creating the keystore.
<code><AndroidSigningKeyPass></code>	The password of the key within the keystore file. This is the value provided to <code>keytool</code> when creating the keystore file and you're asked to enter the keystore password. This is because the default keystore type assumes that the key password and keystore password are identical. This property also supports <code>env:</code> and <code>file:</code> prefixes that can be used to specify an environment variable or file that contains the password. These options provide a way to prevent the password from appearing in build logs.
<code><AndroidSigningKeyStore></code>	The filename of the keystore file created by <code>keytool</code> . This is the <code>keytool -keystore</code> value used when creating the keystore.
<code><AndroidSigningStorePass></code>	The password for the keystore file. This is the value provided to <code>keytool</code> when creating the keystore file and you're asked to enter the keystore password. This is because the default keystore type assumes that the keystore password and key password are identical. This property also supports <code>env:</code> and <code>file:</code> prefixes that can be used to specify an environment variable or file that contains the password. These options provide a way to prevent the password from appearing in build logs.
<code><PublishTrimmed></code>	A boolean value that indicates whether the app should be trimmed. The default value is <code>true</code> for release builds.

For a full list of build properties, see [Build properties](#).

Important

Values for these build properties don't have to be provided in the project file. They can also be provided on the command line when you publish the app. This enables you to omit specific values from your project file.

The following example shows a typical property group for building and signing your Android app:

XML

```
<PropertyGroup Condition="$ (TargetFramework.Contains('-android')) and '$ (Configuration)' == 'Release'">
```

```
<AndroidSigningKeyStore>myapp.keystore</AndroidSigningKeyStore>  
<AndroidSigningKeyAlias>myapp</AndroidSigningKeyAlias>  
</PropertyGroup>
```

This example `<PropertyGroup>` adds a condition check, preventing those settings from being processed unless the condition check passes. The condition check looks for two things:

1. The target framework is set to something containing the text `-android`.
2. The build configuration is set to `Release`.

If either of those conditions fail, the settings aren't processed. More importantly, the `<AndroidSigningKeyStore>` and `<AndroidSigningKeyAlias>` settings aren't set, preventing the app from being signed.

For security reasons, you shouldn't supply a value for `<AndroidSigningKeyPass>` and `<AndroidSigningStorePass>` in the project file. You can provide these values on the command line when you publish the app, or use the `env:` or `file:` prefixes to prevent the password from appearing in build logs. For example, to use an environment variable named `AndroidSigningPassword`:

XML

```
<PropertyGroup Condition="$(TargetFramework.Contains('-android')) and  
    '$(Configuration)' == 'Release'">  
    <AndroidSigningKeyStore>myapp.keystore</AndroidSigningKeyStore>  
    <AndroidSigningKeyAlias>myapp</AndroidSigningKeyAlias>  
    <AndroidSigningKeyPass>env:AndroidSigningPass-  
word</AndroidSigningKeyPass>  
    <AndroidSigningStorePass>env:AndroidSigningPass-  
word</AndroidSigningStorePass>  
</PropertyGroup>
```

Important

The `env:` prefix isn't supported when `$(AndroidPackageFormat)` is set to `aab`.

Alternatively, to use a file located at `C:\Users\user1\AndroidSigningPassword.txt`:

XML

```
<PropertyGroup Condition="$(TargetFramework.Contains('-android')) and  
    '$(Configuration)' == 'Release'">  
    <AndroidSigningKeyStore>myapp.keystore</AndroidSigningKeyStore>  
    <AndroidSigningKeyAlias>key</AndroidSigningKeyAlias>
```



```
<AndroidSigningKeyPass>file:C:\Users\user1\AndroidSigningPass-  
word.txt</AndroidSigningKeyPass>  
  <AndroidSigningStorePass>file:C:\Users\user1\AndroidSigningPass-  
word.txt</AndroidSigningStorePass>  
</PropertyGroup>
```

Distribute the app

The signed APK or AAB file can be distributed with one of the following approaches:

- The most common approach to distributing Android apps to users is through Google Play. Google Play requires that you submit your app as an *Android App Bundle* (AAB). For more information, see [Upload your app to the Play Console](#) on developer.android.com
- APK files can be distributed to Android devices through a website or server. When users browse to a download link from their Android device, the file is downloaded. Android automatically starts installing it on the device, if the user has configured their settings to allow the installation of apps from unknown sources. For more information about opting into allowing apps from unknown sources, see [User opt-in for unknown apps and sources](#) on developer.android.com.

See also

- [About Android App Bundles](#) on developer.android.com